

Thermal Prediction Error Analysis and Set-FNO Modeling

Yiwen Hou

1 Thermal Prediction Error Analysis

1.1 Incorrect Parameter Representation

Original implementation stored only global values:

`[n_comp, move_distance]`

Missing critical information:

- Component identity (which one moves)
- Movement direction (dx, dy)
- Final absolute position

Different layouts could map to identical inputs.

Fix: Use per-component representation:

$$[x_1, y_1, p_1, \dots, x_N, y_N, p_N]$$

1.2 Mismatch with Set-FNO Input

Set-FNO expects:

$$(N, n_{comp}, d_{per_comp})$$

Original input was global scalar $\rightarrow$ violates set structure.

Fix: each component has independent feature (x, y, p) .

1.3 Missing Power Information

Original data only stored position:

$$[x, y]$$

But thermal field depends on:

$$T = f(x, y, p)$$

Without power p , different heat sources become indistinguishable.

Fix: add power feature $\rightarrow (x, y, p)$.

1.4 Incorrect Sample Naming

Original naming based on movement distance $\rightarrow$ not unique.

Fix: use absolute coordinates $\rightarrow$ unique physical layout.

2 Power Setup and Modification

2.1 Original Power Setting

Component	Power (W)
U1	2.2
U2	2.5
U3	2.8
U4	3.0
U5	3.2
U6	2.6
U7	2.4
U8	2.9

Power values are fixed and not modified.

2.2 Why Power Caused Generalization Failure

Although power values remain unchanged, the **total power** increases with component count:

- 1–5 components: $P_{total} \approx 2.2\text{--}13.7$ W
- 6–8 components: $P_{total} \approx 16\text{--}23$ W

This leads to:

- Temperature scale shift ($25\text{--}76^\circ\text{C} \rightarrow 80\text{--}100^\circ\text{C}$)
- Model learns power-dependent magnitude
- Severe distribution shift

2.3 Physics-Informed Fix

Temperature is normalized as:

$$T_{norm} = \frac{T - T_{amb}}{P_{total}}$$

This converts temperature into a **thermal impedance map** ($^\circ\text{C}/\text{W}$).

Advantages:

- Removes dependency on total power
- Unifies data scale across component counts
- Makes learning physically consistent

3 Set-FNO Modeling

3.1 Model Architecture

Pipeline:

$$(x, y, p) \rightarrow \text{Set Transformer} \rightarrow \text{FNO} \rightarrow T(x, y)$$

- Set Transformer: handles variable number of components
- FNO: predicts continuous temperature field

- Captures spatial interactions more explicitly than FNO

Upgrade path:

Set Transformer + FNO $\rightarrow$ Set Encoder + GNO

3.2 Results Comparison

Metric	v2	v3
Test R^2 (1–5)	0.9727	0.9910
Gen R^2 (6–8)	-180.87	0.8946
6 comp	-126.20	0.9128
7 comp	-181.02	0.9291
8 comp	-216.82	0.8420

3.3 Key Insight

Before:

- Model learns absolute temperature magnitude
- Fails under power shift

After:

- Model learns thermal impedance distribution
- Generalizes across component counts

4 Model Capability and Future Directions

4.1 Current Capability

The current model (Set Transformer + FNO + physics normalization) achieves:

- Variable number of components (set-based input)
- Accurate thermal field prediction
- Generalization across component counts

4.2 Current Limitations

The model still has the following limitations:

- Cannot generalize to different PCB sizes (geometry OOD)
- Cannot adapt to different material properties (k)
- Cannot handle varying boundary conditions (h)

Summary: The current framework solves a fixed-domain operator learning problem with set input, but does not yet support geometry-aware learning.

4.3 Candidate Models for Improvement

1) Graph Neural Operator (GNO) [Recommended]

- Idea: represent PCB as a graph (nodes = grid points, edges = adjacency)
- Learn mapping: $(\text{graph}, \text{sources}) \rightarrow T$
- Supports arbitrary PCB size and irregular geometry

2) DeepONet

- Branch: component set (x, y, p)
- Trunk: spatial coordinates (x, y)
- Learns $T(x, y) = G(\text{sources})(x, y)$
- Supports continuous domain and varying PCB size
- Useful as a comparison baseline

3) CNN-based Conditional Model (U-Net)

- Convert components to heatmap input
- Apply CNN to predict temperature
- Simple and practical
- Limited generalization (not truly set-based)

4.4 PCB Size Generalization

Why current model fails:

- Fixed grid (100×100)
- FNO assumes fixed domain and resolution

Possible solutions:

- Coordinate normalization:

$$(x, y) \rightarrow (x/L_x, y/L_y)$$

- Include PCB size (L_x, L_y) as input
- Use coordinate embedding
- Replace FNO with GNO (recommended)

4.5 Recommended Development Path

- Step 1: Set-FNO with physics normalization (completed)
- Step 2: Add DeepONet as baseline
- Step 3: Replace FNO with GNO for geometry-aware learning
- Step 4: Extend to variable PCB size, material, and boundary conditions

4.6 Conclusion

While Set-FNO successfully handles variable component configurations, it is limited to fixed spatial domains. Future work will focus on improving input representation and enabling geometry-aware operator learning for varying PCB sizes and physical conditions.